

```

# PolyFuse V2
## A Standalone, End-to-End Analytical CNN Layer Fusion Optimizer for Spatial Accelerators

**The Goal**: Minimizing energy and latency in Deep Neural Network hardware acceleration by

*An In-Depth Presentation of the PolyFuse V2 Architecture*
<!-- slide -->
# System Interface & Data Ingestion

PolyFuse V2 operates as a standalone CLI tool that takes Timeloop-format inputs and produces

### Inputs (Timeloop Format)
* **Architecture (`arch.yaml`)**: Defines the full memory hierarchy, PE array dimensions,
* **Components (`components/`)**: Defines energy-per-action tables for every hardware primitive
* **Network (`layers/`)**: A linear chain of problem YAMLS defining convolution dimensions

### Core Principle
All hardware parameters must be **parsed** dynamically. There are no hardcoded hardware constraints
<!-- slide -->
# Hardware Abstraction Layer (HAL)

Before optimization begins, the HAL interprets the target architecture:

* **Buffer Inventory**: Extracts buffer properties at each storage level (name, memory depth)
* **Capacity Calculation**: Dynamically computes capacity per bank in words.
* **Buffer Role Assignment**: Determines which buffers hold Inputs, Weights, and Outputs
* **PE Array Geometry**: Extracts spatial bounds (`meshX`, `meshY`) to determine the processing area
<!-- slide -->
# The Three-Stage Core Architecture

PolyFuse V2 is structured into three consecutive analytical stages that guarantee a mathematically optimal solution.

1. **Stage 1: Zero-Search Algebraic Tiling**
   Determines mathematically optimal tile sizes without simulation or iterative search.
2. **Stage 2: Hardware-Aware Spatial Routing (MIQP)**
   Maps valid layer stacks spatially across the NoC while ensuring data causality.
3. **Stage 3: Dynamic Programming (DP) Partitioning**
   Finds the globally optimal partition of the full linear chain into fused stacks.
<!-- slide -->
# Stage 1: Zero-Search Algebraic Tiling

Propagates required input footprints backward from the output tile dimension using an exact formula:


$$T_{in}^{(i)} = (T_{out}^{(i)} - 1) \times S^{(i)} + R^{(i)}$$


### Key Mechanisms:

```

```

*   **Multi-Level Memory**: Solves deeper hierarchies (L2 → L1) using decoupled, cascaded qu
*   **Complex Topologies**: For networks with skip connections, replaces quadratic formulas
<!-- slide -->
# Stage 1 (Cont.): Strict Buffer Constraints

For every layer in a fused stack, all three constraints must hold simultaneously:

1.   **Input Buffer**:  $[T_{in}^{(i)}]^2 \times C^{(i)} \leq \text{InputBuffer}_{\text{capacity}}$ 
2.   **Output Buffer**:  $[T_{out}^{(i)}]^2 \times M^{(i)} \leq \text{AccuBuffer}_{\text{capacity}}$ 
3.   **Weight Buffer (if cached)**: Uses a greedy knapsack algorithm to pack as many weights

> [!CAUTION]
> No receptive field truncation! Intermediate tile sizes must never be shrunk to fit. If the
<!-- slide -->
# Stage 2: Hardware-Aware Spatial Routing

Optimizes the physical spatial allocation (X, Y) of operations on the Processing Element (PE)

Routing Penalty - Convex Quadratic Programming
Minimizes physical NoC hop distances using a convex squared-distance objective:

$$J = \min_{X, Y} \sum \left( X^{(i+1)} - X^{(i)} \right)^2 + \left( Y^{(i+1)} - Y^{(i)} \right)^2$$


This formulation penalizes long hops quadratically (distance 4 costs 16x) and solves in under 100ms
<!-- slide -->
# Stage 2 (Cont.): Causality & Extreme Vectors

Instead of using standard Farkas' Lemma which results in 50,000+ variables, PolyFuse V2 uses a
causality matrix to enforce constraints.

For a  $3 \times 3$  kernel, causality is enforced only at the 4 geometric corner vectors:

$$\Theta_2 \cdot \vec{x} - \Theta_1 \cdot (\vec{x} - \vec{d}_{ext}) \geq 1 \quad \forall \vec{d}_{ext} \in \{ \vec{d}_1, \vec{d}_2, \vec{d}_3, \vec{d}_4 \}$$


> [!TIP]
> This reduces constraints from thousands of variables down to ~4-8 linear equations per layer
<!-- slide -->
# Stage 3: Dynamic Programming Partitioner

Runs an  $O(N^2)$  pass to find the absolute globally optimal partition of layers.


$$DP[i] = \min_{0 \leq j < i} \left( DP[j] + \text{EnergyCost}(\text{Stack}(j, i)) \right)$$


### Optimization Objectives:
*   Energy: Minimizes total energy ( $E_{MAC} + E_{SRAM} + E_{DRAM}$ )
*   Latency: Minimizes cycles, using analytical roofline modeling for compute-bound vs memory-bound
*   EDP: Minimizes the Energy-Delay Product ( $E_{total} \times \text{Latency}$ )
<!-- slide -->
# Stage 3 (Cont.): Recompute vs. Store

```

For each fused stack boundary, the algorithm analytically decides whether to recompute the c

```
```python
if (Vol_halo * Cost_MAC) < (Vol_spill * Cost_DRAM):
 # Recompute (use Index Set Splitting to decouple tiles)
else:
 # Store (share halo data sequentially through SRAM)
```
```

****Timeloop Verification****: Finally, PolyFuse V2 invokes ``timeloop-mapper`` as a subprocess for
<!-- slide -->
Prior Works: Known Pitfalls & Resolutions

PolyFuse V2 was built to explicitly avoid bugs present in previous heuristic approaches (e.g.

- * ****The Overwrite Bug****: Previous tools used ``=`` to calculate weight footprints, hallucinating
- * ****Receptive Field Truncation****: Previous tools silently broke convolution physics by sh
- * ****Buffer Collisions****: PolyFuse treats Spatial Hardware buffers as physically disjoint

<!-- slide -->
Validation & Benchmarks

PolyFuse V2 automatically evaluates against the ****Naive Baseline**** (executing one layer at a

****Target Networks****:

- * ****AlexNet**** & ****VGG02**** (Deep chains)
- * ****MobileNetV2**** (Depthwise separable convolutions)
- * ****ResNet-50**** & ****EfficientNet-B0****

****Target Architectures****:

- * ****Simba**** (Chiplet NoC, Weight-Stationary)
- * ****Eyeriss**** (Spatial array, Row-Stationary)
- * ****Gemmini**** (Systolic array, Configurable)